DOCUMENTAÇÃO TÉCNICA

I6 - Detector de Códigos Gráficos

Leitura de QR Code e códigos de barras com integração P4 via socket TCP e publicação Kafka

```
Produção:  P4 -> Socket TCP -> I6 -> Kafka -> A2
Teste:     Câmera IP -> I6 -> Kafka -> A2
```

Versão documentada: DetectorQR_I6_socket_v2

Outubro de 2026

Sumário

1. Visão geral e objetivo
2. Responsabilidades e limites do I6
3. Arquitetura e fluxo de dados
4. Estrutura do projeto
5. Dependências e pré-requisitos
6. Configuração do .env
7. Como executar do zero - modo teste
8. Como testar o Kafka
9. Como testar o caminho socket com mock_p4.py
10. Modo produção com P4
11. Protocolo TCP preparado no I6
12. Contrato de entrada ods.p4.frame
13. Processamento e detecção
14. Estabilidade e múltiplas câmeras
15. Contrato de saída ods.i6.code_read
16. Kafka no I6
17. Docker e Docker Compose
18. Logs, diagnóstico e problemas comuns
19. Checklist de integração
20. Pontos ainda dependentes do P4

Apêndice A. Comandos rápidos

Apêndice B. Referência das variáveis de ambiente

1. Visão geral e objetivo

O I6 é o componente responsável por localizar e decodificar códigos gráficos presentes nos frames de vídeo e publicar cada leitura válida para o restante do sistema ODS. Na versão atual, o componente reconhece QR Code, CODE 128 e EAN-13, preserva a origem do frame e não interpreta semanticamente o conteúdo lido.

O projeto foi estruturado para operar em dois modos. No modo de teste, o próprio I6 abre uma ou mais câmeras IP diretamente. No modo de produção, o I6 não controla câmeras físicas: ele recebe frames do P4 por uma conexão preparada para socket TCP. Após a leitura, o I6 publica eventos no Kafka no tópico ods.i6.code_read.

Importante: A interface P4 -> I6 foi preparada para socket TCP porque esta é a direção informada para a integração. O enquadramento exato dos bytes no socket ainda é provisório e deve ser confirmado com a equipe do P4 antes da integração final.

2. Responsabilidades e limites do I6

- **O I6 faz:** recebe frames, valida os metadados, detecta/decodifica códigos, aplica estabilidade de leitura e publica ods.i6.code_read no Kafka.
- **O I6 não faz:** captura física de câmera em produção, controle de catraca, validação de CPF, regras de entrada/saída ou interpretação de negócio do conteúdo do QR.
- **Em teste:** o I6 pode abrir câmera IP diretamente apenas para permitir desenvolvimento independente do P4.
- **Em produção:** a câmera e sua captura pertencem ao P4; o I6 é consumidor do stream.
- **Qualidade de câmera:** não faz parte desta implementação do I6.

3. Arquitetura e fluxo de dados

3.1. Fluxo de produção

```
Câmera física
   |
   v
  P4
   |
   |  socket TCP: metadados ods.p4.frame + buffer do frame
   v
  I6
   |  validação -> detecção -> estabilidade
   |
   |  Kafka / tópico ods.i6.code_read
   v
  A2
```

O Kafka transporta o evento textual de leitura produzido pelo I6. O vídeo não é enviado ao Kafka por esta implementação.

3.2. Fluxo de teste direto

```
Câmera IP (uma ou várias)
   |
   v
  I6
   |  OpenCV -> pyzbar -> estabilidade
   |
   |  Kafka / ods.i6.code_read
   v
  A2 ou consumidor de teste
```

4. Estrutura do projeto

```
DetectorQR_I6_socket_v2/
|-- leitor.py
|-- .env.example
|-- Dockerfile
|-- docker-compose.yml
|-- requirements.txt
|-- .dockerignore
|-- teste_consumer_i6_kafka.py
|-- mock_p4.py
`-- COMO_RODAR.txt
```

Arquivo	Função
leitor.py	Aplicação principal do I6. Contém configuração, socket, detecção, estabilidade, Kafka e os dois modos de execução.
.env.example	Modelo das variáveis de ambiente. Deve ser copiado para .env antes de executar com Docker Compose.
Dockerfile	Cria a imagem Python 3.11, instala libzbar0 e as dependências Python e inicia leitor.py.
docker-compose.yml	Sobe o serviço i6 usando o .env.
requirements.txt	numpy, opencv-contrib-python-headless, pyzbar e confluent-kafka.
teste_consumer_i6_kafka.py	Consumidor auxiliar para visualizar os eventos publicados no Kafka.
mock_p4.py	Simulador da saída socket do P4. Usado apenas para testar o lado I6 antes da integração real.
COMO_RODAR.txt	Resumo operacional dos comandos de execução.

5. Dependências e pré-requisitos

5.1. Caminho recomendado: Docker

- Docker Desktop (Windows/macOS) ou Docker Engine + Docker Compose Plugin (Linux).
- Acesso de rede ao broker Kafka configurado.
- Para modo teste: URL de uma câmera IP acessível a partir do container.
- Para produção: host e porta do socket do P4 acessíveis a partir do container.

5.2. Dependências instaladas na imagem

Dependência	Uso
Python 3.11	Execução do leitor.
libzbar0	Biblioteca nativa usada pelo pyzbar para decodificação.
NumPy	Representação e reconstrução de buffers de imagem.
OpenCV headless	Captura de stream IP em teste, conversões de imagem e binarização.
pyzbar	Decodificação de QR Code e códigos de barras.
confluent-kafka	Producer Kafka do evento ods.i6.code_read.

O módulo socket e o módulo struct usados na integração TCP fazem parte da biblioteca padrão do Python e não aparecem em requirements.txt.

6. Configuração do .env

Antes de executar o Compose, crie o arquivo .env a partir de .env.example.

```
# Windows CMD
copy .env.example .env

# PowerShell
Copy-Item .env.example .env

# Linux/macOS
cp .env.example .env
```

Variável	Padrão/exemplo	Finalidade
I6_TEST_MODE	true	true abre câmeras IP diretamente; false usa o caminho P4 via socket TCP.
KAFKA_BOOTSTRAP_SERVERS	34.66.235.91:9093	Broker Kafka acessado pelo producer e pelo consumidor de teste.
KAFKA_TOPIC	ods.i6.code_read	Tópico no qual o I6 publica leituras.
I6_ARTIFACT_VERSION	i6-qrcode-reader-2.0	Versão do artefato gravada nos eventos.
I6_TEST_CAMERAS_JSON	[{"source_id":"cam0",...}]	Lista de câmeras IP no modo teste.
I6_TEST_RECONNECT_SECONDS	2.0	Intervalo entre tentativas de reconexão da câmera de teste.
P4_HOST	p4	Host/DNS/IP do servidor TCP do P4.
P4_PORT	5000	Porta TCP esperada no P4.
P4_CONNECT_RETRY_SECONDS	2.0	Intervalo de reconexão caso o P4 esteja indisponível.
I6_ACQUIRE_CONFIRM_FRAMES	4	Número de confirmações do candidato para assumir um código como ativo.
I6_SWITCH_CONFIRM_FRAMES	10	Confirmações necessárias para trocar o código ativo por outro.
I6_SWITCH_AREA_RATIO	1.5	Novo código precisa ter área mínima relativa ao ativo para iniciar troca.
I6_QR_LOST_SECONDS	3.0	Tempo sem leitura para considerar o código ativo perdido.
I6_REPEAT_COOLDOWN_SECONDS	3.0	Intervalo mínimo para republicar o mesmo código ativo.

7. Como executar do zero - modo teste

Este é o caminho mais simples para validar o DetectorQR sem depender do P4.

7.1. Passo 1 - entrar na pasta

```
cd DetectorQR_I6_socket_v2
```

7.2. Passo 2 - criar o .env

```
copy .env.example .env
```

7.3. Passo 3 - configurar a câmera

Edite .env e mantenha:

```
I6_TEST_MODE=true
I6_TEST_CAMERAS_JSON=[{"source_id":"cam0","url":"http://SEU_IP:PORTA/video"}]
```

Para duas câmeras:

```
I6_TEST_CAMERAS_JSON=[{"source_id":"cam0","url":"http://IP1:PORTA/video"},{"source_id":"cam1","url":"htt
p://IP2:PORTA/video"}]
```

7.4. Passo 4 - conferir Kafka

Confirme se KAFKA_BOOTSTRAP_SERVERS e KAFKA_TOPIC apontam para a infraestrutura correta. O valor atual do projeto é:

```
KAFKA_BOOTSTRAP_SERVERS=34.66.235.91:9093
KAFKA_TOPIC=ods.i6.code_read
```

7.5. Passo 5 - construir e iniciar

```
docker compose up --build
```

No primeiro build o Docker instala as dependências. Em execuções seguintes, se nada estrutural mudou, pode usar apenas docker compose up.

7.6. O que deve aparecer nos logs

```
============================================================
I6 - Detector de códigos gráficos
I6_TEST_MODE=True
============================================================
[Kafka] produtor pronto.
[Kafka] tópico: ods.i6.code_read
[I6] MODO TESTE
[TESTE] conectando cam0 -> http://...
[TESTE] cam0 conectada.
```

Ao mostrar um código válido, após a lógica de estabilidade, o I6 imprime o JSON publicado e o callback do Kafka confirma a entrega.

8. Como testar o Kafka

O arquivo teste_consumer_i6_kafka.py é um consumidor auxiliar. Ele não faz parte da execução do container do I6; serve para conferir o que chegou ao tópico.

8.1. Preparar um ambiente Python local

```
python -m venv .venv

# Windows
.venv\Scripts\activate

# Linux/macOS
source .venv/bin/activate

pip install confluent-kafka
```

8.2. Executar o consumidor

```
python teste_consumer_i6_kafka.py
```

O consumidor usa o mesmo KAFKA_BOOTSTRAP_SERVERS e KAFKA_TOPIC do ambiente. Ao receber uma mensagem, imprime o evento completo em JSON.

Observação: Se o Kafka não estiver acessível a partir do computador ou container, o detector pode continuar processando frames, mas a publicação não será concluída corretamente. Verifique rede, DNS, firewall e endereço do broker.

9. Como testar o caminho socket com mock_p4.py

Esse teste valida o lado de produção do I6 sem depender do P4 real. O mock abre uma fonte de vídeo e atua como servidor TCP na porta 5000 usando o framing provisório esperado pelo I6.

9.1. Instalar OpenCV no host

```
pip install opencv-contrib-python-headless
```

9.2. Iniciar o mock

```
python mock_p4.py http://IP:PORTA/video cam0
```

9.3. Configurar o I6 para receber pelo socket

Se o mock está rodando no Windows e o I6 está dentro do Docker:

```
I6_TEST_MODE=false
P4_HOST=host.docker.internal
P4_PORT=5000
```

9.4. Subir o I6

```
docker compose up --build
```

```
Câmera IP -> mock_p4.py -> TCP:5000 -> leitor.py -> Kafka
```

Escopo do mock: mock_p4.py não representa a implementação interna do P4. Ele apenas simula a saída que o I6 espera receber pelo socket.

10. Modo produção com P4

Quando a equipe do P4 disponibilizar o servidor socket, altere apenas as variáveis do .env e use I6_TEST_MODE=false.

```
I6_TEST_MODE=false
P4_HOST=p4
P4_PORT=5000
P4_CONNECT_RETRY_SECONDS=2.0
```

O valor p4 funciona quando o P4 estiver em um serviço/container acessível pelo nome p4 na mesma rede Docker. Se o P4 estiver em outra máquina, use o IP ou DNS dessa máquina.

Ao iniciar, o I6 executa socket.create_connection((P4_HOST, P4_PORT), timeout=5.0). Após conectar, remove o timeout de leitura e aguarda continuamente os frames. Se a conexão cair, espera P4_CONNECT_RETRY_SECONDS e tenta novamente.

11. Protocolo TCP preparado no I6

TCP não preserva fronteiras de mensagens. Por isso o I6 não pode assumir que um recv() corresponde exatamente a um JSON ou a uma imagem. A função receber_exatamente() acumula bytes até atingir a quantidade esperada.

```
[4 bytes]  tamanho do JSON, uint32 big-endian
[N bytes]  JSON UTF-8 ods.p4.frame
[8 bytes]  tamanho da imagem, uint64 big-endian
[M bytes]  buffer bruto do frame
```

O código usa struct.unpack("!I", ...) para o tamanho do JSON e struct.unpack("!Q", ...) para o tamanho do frame. O caractere ! indica ordem de bytes de rede (big-endian).

- **Limite de metadados:** 64 KiB.
- **Limite de frame:** 64 MiB.
- **Proteção de sincronismo:** se tamanho ou contrato forem inválidos, a conexão é encerrada e refeita.
- **Conversão:** o buffer recebido é reconstruído e convertido para BGR antes da detecção.

Provisório: Esses 4 + JSON + 8 + frame são o protocolo preparado no I6, não uma especificação já confirmada do P4. Quando o P4 definir seu framing, a principal função a revisar é receber_frame_p4().

12. Contrato de entrada ods.p4.frame

No modo produção, o I6 valida os metadados antes de aceitar o frame.

```
{
 "schema": "ods.p4.frame",
 "producer": "P4",
 "captured_at": "2026-09-23T14:29:59.870Z",
 "source_id": "cam0",
 "session_id": "9c2a1f3e-4d5b-4c6a-9e8f-0a1b2c3d4e5f",
 "payload": {
  "frame_index": 48213,
  "image_width": 1920,
```

```
    "image_height": 1080,
    "pixel_format": "BGR",
    "fps": 30.0,
    "raw": true
  }
}
```

Campo	Validação realizada pelo I6
schema	Deve ser exatamente ods.p4.frame.
producer	Deve ser P4.
captured_at	ISO 8601 UTC terminando em Z.
source_id	String não vazia; identifica a origem/câmera.
session_id	UUID válido.
frame_index	Inteiro maior ou igual a zero.
image_width / image_height	Inteiros positivos.
pixel_format	NV12, RGB, BGR ou GRAY.
fps	Número maior que zero.
raw	Deve ser true.

O tamanho esperado do buffer também é calculado. BGR/RGB usam largura x altura x 3 bytes, GRAY usa largura x altura e NV12 usa largura x altura x 3/2.

13. Processamento e detecção

1. O frame chega ao processar_frame(frame, metadados).
2. O estado da fonte é obtido por source_id e session_id.
3. Frames repetidos ou fora de ordem (frame_index menor ou igual ao último) são descartados.
4. O detector utiliza o canal azul do frame.
5. Aplica binarização de Otsu.
6. O pyzbar procura códigos na imagem binarizada.
7. Somente QRCODE, CODE128 e EAN13 são convertidos para QR_CODE, CODE_128 e EAN_13.
8. decoded_data é decodificado como UTF-8 e publicado sem interpretação de negócio.
9. A bounding box é limitada às dimensões do frame.
10. A leitura passa pela lógica de estabilidade antes de ser publicada no Kafka.

Se o conteúdo decodificado não for UTF-8 válido, a implementação atual ignora aquela leitura porque o contrato de saída exige string.

14. Estabilidade e múltiplas câmeras

O dicionário estados mantém um estado independente para cada source_id. Assim, cam0, cam1 e cam2 podem compartilhar o mesmo processo sem misturar cooldown, código ativo ou frame_index.

Parâmetro	Padrão	Comportamento
I6_ACQUIRE_CONFIRM_FRAMES	4	O candidato precisa atingir 4 confirmações para se tornar ativo.
I6_SWITCH_CONFIRM_FRAMES	10	Outro código precisa atingir 10 confirmações para substituir o atual.
I6_SWITCH_AREA_RATIO	1.5	O novo código deve ter área pelo menos 1,5x a área ativa para iniciar a troca.

I6_QR_LOST_SECONDS	3.0 s	Sem leitura por esse período, o código ativo é limpo.
I6_REPEAT_COOLDOWN_SECONDS	3.0 s	O mesmo código ativo só é republicado após esse intervalo.

Quando existem várias leituras no mesmo frame, a implementação atual seleciona a de maior área como leitura predominante. Uma mudança de session_id reinicia somente o estado daquela fonte.

15. Contrato de saída ods.i6.code_read

Quando uma leitura é considerada estável, o I6 monta, valida e publica o seguinte perfil de evento:

```
{
 "message_type": "event",
 "message_id": "UUID",
 "schema": "ods.i6.code_read",
 "schema_version": "1.0",
 "producer": "I6",
 "published_at": "2026-10-29T12:00:00.000Z",
 "captured_at": "2026-10-29T11:59:59.900Z",
 "source_id": "cam0",
 "session_id": "UUID",
 "sequence": 0,
 "artifact_version": "i6-qrcode-reader-2.0",
 "payload": {
  "frame_index": 48213,
  "code_type": "QR_CODE",
  "decoded_data": "conteudo-do-codigo",
  "bbox": [120, 80, 420, 380]
 }
}
```

Campo	Origem/significado
message_id	UUID novo criado pelo I6 para a mensagem.
published_at	Instante UTC em que o I6 publica o evento.
captured_at	Preservado do frame recebido do P4; em teste é gerado no I6.
source_id	Origem/câmera da leitura.
session_id	Sessão de captura; preservada do P4.
sequence	Contador crescente mantido pelo I6 por session_id.
frame_index	Índice do frame no qual o código foi detectado.
code_type	QR_CODE, CODE_128 ou EAN_13.
decoded_data	Conteúdo decodificado sem regra de negócio.
bbox	[x_min, y_min, x_max, y_max] do código no frame.

16. Kafka no I6

O Kafka é utilizado apenas na saída do I6. O componente cria um Producer do confluent-kafka com idempotência habilitada e acks=all.

```
Producer({
   "bootstrap.servers": KAFKA_BOOTSTRAP_SERVERS,
```

```
    "enable.idempotence": True,
    "acks": "all"
})
```

Cada evento é publicado no tópico KAFKA_TOPIC, atualmente ods.i6.code_read. A chave Kafka é source_id, o que ajuda a manter mensagens da mesma fonte associadas à mesma chave de particionamento.

- **Valor Kafka:** JSON UTF-8 compacto.
- **Chave Kafka:** source_id em UTF-8.
- **Callback:** imprime erro ou confirmação com tópico, partição e offset.
- **Encerramento:** producer.flush(10) tenta concluir mensagens pendentes ao sair.

17. Docker e Docker Compose

17.1. Dockerfile

A imagem usa python:3.11-slim-bookworm, instala libzbar0 e ca-certificates, instala requirements.txt e copia apenas leitor.py para /app.

```
docker compose build
docker compose up

# ou em um único comando
docker compose up --build
```

17.2. Compose

O serviço se chama i6, usa a imagem ods-i6:2.0, recebe as variáveis de .env, reinicia automaticamente em falhas e usa init=true. Não há ports no Compose porque o I6 não oferece um servidor de rede; no caminho de produção ele é cliente do socket do P4 e cliente do Kafka.

17.3. Rodar em segundo plano

```
docker compose up -d --build
docker logs -f ods-i6
```

17.4. Parar

```
docker compose down
```

18. Logs, diagnóstico e problemas comuns

Sintoma	Causa provável / ação
[TESTE] falha ao abrir cam0	URL incorreta, câmera inacessível do container, credenciais erradas ou stream indisponível.
frame repetido/fora de ordem descartado	frame_index recebido não avançou para aquela source_id/session_id.
[P4] conexão indisponível/perdida	P4 não está ouvindo no host/porta, rede não acessível ou conexão caiu. O I6 tenta reconectar.
Tamanho dos metadados/frame inválido	Framing TCP recebido não corresponde ao provisório ou dados foram corrompidos/desalinhados.
Tamanho informado pelo socket não bate com ods.p4.frame	Largura, altura, pixel_format ou tamanho do buffer não correspondem entre si.

producer inválido / schema inválido	O JSON recebido não está no contrato esperado de ods.p4.frame.
Nenhum evento Kafka aparece	Código ainda não atingiu estabilidade, broker/tópico incorreto ou rede Kafka indisponível.
QR detecta visualmente mas não publica	Pode estar em cooldown, aguardando confirmações, conteúdo não UTF-8, tipo não suportado ou o código predominante mudou.
P4_HOST=p4 não resolve	Esse nome só funciona se p4 estiver resolvível na rede Docker. Use IP/DNS real ou host.docker.internal quando apropriado.

18.1. Comandos de inspeção úteis

```
docker ps
docker logs -f ods-i6
docker compose config
docker compose down
docker compose up --build
```

19. Checklist de integração

- O .env existe e está sendo carregado pelo Compose.
- I6_TEST_MODE está correto para o cenário.
- KAFKA_BOOTSTRAP_SERVERS é acessível do container.
- KAFKA_TOPIC está correto.
- Em teste, I6_TEST_CAMERAS_JSON possui source_id únicos e URLs válidas.
- Em produção, P4_HOST e P4_PORT apontam para o servidor do P4.
- O P4 envia ods.p4.frame com source_id, session_id, frame_index, dimensões, pixel_format, fps e raw=true.
- O framing TCP foi confirmado entre P4 e I6.
- O tamanho do buffer corresponde ao pixel_format e às dimensões.
- O consumidor de teste consegue ver ods.i6.code_read no Kafka.
- Múltiplas fontes mantêm source_id distintos.
- A2 recebe decoded_data sem depender de interpretação no I6.

20. Pontos ainda dependentes do P4

O I6 está preparado para ser cliente TCP, mas a integração definitiva depende de informações que pertencem à equipe P4. Antes de considerar a integração encerrada, devem ser confirmados:

- TCP como protocolo final, em vez de outro mecanismo.
- Quem atua como servidor e qual host/porta devem ser usados.
- Framing definitivo: como o receptor distingue metadados e frame dentro do fluxo TCP.
- Formato efetivo do buffer: BGR, RGB, NV12 ou GRAY.
- Comportamento com múltiplas câmeras no mesmo socket.
- Semântica de session_id e frame_index na implementação final.
- Política de reconexão e reinício de sessão após queda.

Impacto esperado: Se apenas o framing do socket mudar, a maior parte do I6 permanece intacta. A adaptação fica concentrada em receber_frame_p4() e, se necessário, na configuração de conexão.

Apêndice A. Comandos rápidos

A.1. Teste direto com câmera IP

```
copy .env.example .env
# editar .env: I6_TEST_MODE=true e I6_TEST_CAMERAS_JSON=...
docker compose up --build
```

A.2. Consumidor Kafka

```
pip install confluent-kafka
python teste_consumer_i6_kafka.py
```

A.3. Teste do caminho socket

```
# terminal 1
python mock_p4.py http://IP:PORTA/video cam0

# .env do I6
I6_TEST_MODE=false
P4_HOST=host.docker.internal
P4_PORT=5000

# terminal 2
docker compose up --build
```

A.4. Produção

```
I6_TEST_MODE=false
P4_HOST=<host do P4>
P4_PORT=<porta do P4>
docker compose up -d --build
docker logs -f ods-i6
```

Apêndice B. Referência das variáveis de ambiente

Variável	Obrigatória em	Descrição
I6_TEST_MODE	Sempre	Seleciona câmera IP direta ou P4/socket.
KAFKA_BOOTSTRAP_SERVERS	Sempre	Endereço(s) do broker Kafka.
KAFKA_TOPIC	Sempre	Tópico de publicação.
I6_ARTIFACT_VERSION	Sempre	Versão incluída no evento.
I6_TEST_CAMERAS_JSON	Modo teste	Array JSON de {source_id,url}.
I6_TEST_RECONNECT_SECONDS	Modo teste	Intervalo de reconexão da câmera.
P4_HOST	Produção	Host/IP/DNS do P4.
P4_PORT	Produção	Porta TCP do P4.
P4_CONNECT_RETRY_SECONDS	Produção	Intervalo entre reconexões.
I6_ACQUIRE_CONFIRM_FRAMES	Sempre	Confirmação inicial.
I6_SWITCH_CONFIRM_FRAMES	Sempre	Confirmação de troca.

I6_SWITCH_AREA_RATIO	Sempre	Relação mínima de área para challenger.
I6_QR_LOST_SECONDS	Sempre	Tempo para limpar código perdido.
I6_REPEAT_COOLDOWN_SECONDS	Sempre	Cooldown de republicação do mesmo código.

Referências técnicas do projeto

Repositório P4 consultado para contexto da captura e integração: https://github.com/ods-26-2/p4

Observação: esta documentação descreve o estado atual do I6 e explicita como provisórios os pontos de protocolo ainda não confirmados pelo P4.